

M É M O

Machine Learning

Ce que je retiens — de Python à la descente de gradient

Réflexes transférables pour la suite du cours et le métier d'ingénieure IA

Quatre axes à garder en tête

Le fil rouge de tout ce qui a été vu

1

Le coût

Combien d'opérations ? Un code juste peut être inutilisable à grande échelle.

2

La forme

shape avant tout. La majorité des bugs viennent de dimensions qui ne correspondent pas.

3

Le silence

Les erreurs les plus coûteuses ne plantent pas : elles renvoient un chiffre faux.

4

La question

Le choix de la métrique décide de la conclusion. Toujours savoir ce qu'on mesure.

Penser au coût, pas seulement au résultat

La complexité dit comment le temps de calcul grandit avec la taille des données

Complexité	Comportement	Exemple rencontré
$O(1)$	Constant	Accès à une clé de dictionnaire
$O(\log n)$	Divisé par 2 à chaque étape	Recherche dichotomique
$O(n)$	Proportionnel	Parcours d'une liste
$O(n \log n)$	Optimal pour un tri	Tri fusion
$O(n^2)$	Explose vite	Deux boucles imbriquées

1 million d'éléments

$O(n^2)$
plusieurs heures

$O(n \log n)$
une fraction de seconde

Les deux réflexes qui reviennent partout

Sortir dès qu'on sait (early exit) — inutile de continuer quand la réponse est acquise.
Exploiter ce qu'on sait déjà — liste triée, données déjà dédoublonnées, moitiés déjà triées.

Vectoriser plutôt que boucler

Le prolongement direct du réflexe de coût, dans le monde NumPy

Python pur

```
resultat = []  
for x in liste:  
    resultat.append(x * 2)
```

La boucle tourne dans l'interpréteur

NumPy

```
resultat = tableau * 2
```

Le calcul descend dans du C compilé

× 10

plus rapide (mesuré, 5 M d'éléments)

÷ 10

de mémoire : octets bruts, pas des objets Python

0

boucle visible dans le code

NumPy — les quatre réflexes

Le socle de tout le calcul en machine learning

1

shape, toujours

.shape est l'attribut le plus consulté. La plupart des erreurs viennent de formes qui ne correspondent pas.

`M.shape` → `(3, 4)`

2

Broadcasting

NumPy aligne les formes en partant de la droite. Compatibles si égales, ou si l'une vaut 1.

`(3, 4) + (4,)` ✓

`(3, 4) + (3,)` ✗

`(3, 4) + (3, 1)` ✓

3

axis = l'axe qui disparaît

Une agrégation réduit d'une dimension. `keepdims=True` conserve l'axe à la taille 1 pour rebroadcaster.

`C.mean(axis=0)` → `(4,)`

`C.mean(axis=1, keepdims=True)`

4

Masques booléens

Une comparaison renvoie un tableau. On filtre avec `&`, `|` et `~` — jamais `and/or` — et parenthèses obligatoires.

`a[(a > 3) & (a < 9)]`

Pandas — les gestes qui reviennent

Découvrir un jeu de données, le nettoyer, l'interroger

Découvrir

```
df.shape  
df.head()  
df.info()  
df.describe()  
df.isna().sum()
```

info() dit ce qu'on a, describe() où sont les valeurs aberrantes.

Sélectionner

```
df["col"]           → Series  
df[["a", "b"]]      → DataFrame  
df.loc[2, "nom"]    étiquette  
df.iloc[0:3]        position
```

Le double crochet est une liste : le résultat reste en 2D.

Interroger

```
df[df["age"] > 30]  
df.groupby("ville")["x"].mean()  
df.groupby("x").agg(["mean", "count"])
```

Toujours count à côté de mean : une moyenne sans effectif ne veut rien dire.

Les pièges silencieux des données réelles

Aucun ne provoque d'erreur — tous produisent un résultat faux

● Espaces invisibles

Les exports à format fixe laissent des espaces de fin. "GO " ≠ "GO", et l'affichage ne montre rien.

```
.str.strip() dès le chargement
```

● Filtre qui ne renvoie rien

Un tableau vide se propage : `.mean()` donne nan, `.max()` lève une erreur trois étapes plus loin.

```
tester .size > 0
```

● Moyenne sans effectif

Une moyenne sur 1 élément s'affiche comme les autres. Rien ne signale qu'elle ne veut rien dire.

```
agg(["mean", "count"])
```

● nan ignoré par défaut

`mean()` saute les valeurs manquantes : le dénominateur change sans avertissement.

```
isna().sum() avant tout calcul
```

Valeurs manquantes : pourquoi est-elle absente ?

La bonne question n'est pas « comment je la comble ? »

Absence structurelle

L'absence porte une information : la mesure n'existe pas pour cette catégorie.

- Deux colonnes complémentaires selon la norme appliquée
- Une grandeur nulle par nature, laissée vide

Un `dropna()` global supprimerait une catégorie entière.

Absence accidentelle

La donnée existe mais n'a pas été renseignée.

- Quelques pourcents épars, sans motif
- Aucun lien avec une catégorie

Là, `dropna()` ou `fillna()` se discutent.

Croiser les valeurs manquantes avec une variable catégorielle révèle le motif — et évite de biaiser toute l'analyse.

Le triptyque de l'apprentissage

Trois briques distinctes — le même mécanisme entraîne un réseau de neurones

1

Le modèle

$$f(x) = wx + b$$

Prend un x , suppose w et b connus, produit une prédiction.



2

La fonction de coût

$$J(w, b)$$

Mesure à quel point ce couple (w, b) est mauvais. Moyenne des erreurs au carré.



3

La descente de gradient

$$w \leftarrow w - \alpha \partial J / \partial w$$

Ajuste w et b pas à pas dans le sens de la pente, jusqu'au minimum.

En pratique : `LinearRegression().fit(X, y)` condense les trois. Comprendre le mécanisme reste indispensable — il se généralise.

Le taux d'apprentissage vit dans une fenêtre étroite

Trop petit : on n'arrive jamais. Trop grand : tout diverge en nan.



$\alpha = 0,001$

20 000 itérations et toujours pas arrivé. Ça avance, mais dix fois trop lentement.

$\alpha = 0,46$

Le coût explose au lieu de baisser : inf, puis nan. Un nan contamine tout ce qui suit.

Tracer le coût en fonction des itérations est le diagnostic de base : il doit décroître à chaque pas.

La normalisation ne sert pas qu'au multi-variables

Centrer x découple w et b — la vallée étroite redevient un bol

Sans normalisation

x n'est pas centré : augmenter w relève toutes les prédictions, b doit compenser. Les deux paramètres sont couplés — le coût forme une vallée longue et étroite.

Avec normalisation

$(x - \mu) / \sigma \rightarrow$ les directions deviennent indépendantes, la descente va tout droit. Et un α bien plus grand devient utilisable sans diverger.

> 20 000

itérations sans normaliser



< 200

itérations après normalisation, à α identique

Conserver μ et σ : les paramètres appris portent sur la variable transformée. Il faut les reconvertir pour retrouver des unités interprétables.

Évaluer un modèle : combien, et surtout où

Un score global ne dit jamais où le modèle échoue

R^2

Part de la variation expliquée. En régression simple, c'est exactement le carré de la corrélation.

MAE / RMSE

L'erreur moyenne, dans l'unité de la cible.
Directement interprétable, contrairement au coût J.

Les résidus

Le graphique le plus informatif. Des erreurs qui s'évasent ou une traînée asymétrique désignent une variable manquante.

Le calcul du R^2 dépend de la population choisie

Mélanger deux sous-populations brouille la relation ; isoler une catégorie peut la révéler beaucoup plus nette. Un même indicateur calculé sur deux périmètres différents n'est pas comparable.

Corrélation, pente, causalité

Trois notions souvent confondues

1

La corrélation

Mesure l'alignement des points. Sans unité, toujours entre -1 et 1 , insensible au changement d'échelle.

2

La pente

Mesure de combien y varie quand x augmente d'une unité. A une unité, dépend des échelles choisies.

3

Ni l'une ni l'autre n'est une cause

Une corrélation forte peut refléter une identité physique, une variable cachée, ou une coïncidence.

Une corrélation faible ne signifie pas absence de relation : le coefficient ne mesure que la linéarité. Toujours regarder le nuage de points.

Hygiène de travail

Ce qui distingue un brouillon d'un livrable

Environnement

- Un dossier = un projet = un .venv
- Ouvrir le projet, pas le dossier parent
- which python quand un import échoue
- requirements.txt : dépendances directes

Notebook

- Restart and Run All avant chaque commit
- Une variable réutilisée = état fantôme
- Numéros d'exécution dans l'ordre
- Chaque chiffre affiché doit être mesuré

Git & portfolio

- add / commit / push, à chaque étape finie
- .venv ignoré, jamais versionné
- README : question, méthode, résultat, limites
- Nommer les limites de son analyse

Le fil rouge

Un résultat juste ne suffit pas

- Il doit être calculable à l'échelle réelle — d'où le coût et la vectorisation.
- Il doit être vérifié, car l'erreur la plus dangereuse ne plante pas.
- Il doit être situé : sur quelle population, avec quelle métrique, dans quelles limites.
- Il doit être reproductible par quelqu'un d'autre.

Prochaine étape : régression multiple — plusieurs variables, et la normalisation devient obligatoire.